

Coding test

History of Manual Adjustments to an Earning Line

Alcor OS is an all-in-one platform for managing contractors for product companies. They've contracted you to create a proof of concept for their new payroll platform.

The company has the following **business case** you will work on:

- A payroll specialist reviews a line with an employee's base salary that the system calculates automatically.
- Sometimes the specialist needs to manually correct it (e.g. because of a declined benefit or a late correction).
- The correction is entered as a [positive or negative] amount with a mandatory comment explaining why.

Your task is to design and implement a domain model that satisfies business case and follows these rules:

- A single line can receive multiple corrections over time:
 - Each one must stay visible and traceable - no correction may ever be edited or silently deleted once saved.
 - If the specialist made a mistake, they add a new correction that fixes the previous one.
- Once a line has received at least one manual correction, automatic system recalculation must no longer affect it — the specialist's corrections take permanent precedence, even if the underlying source data used to calculate the line later changes.
- At any point, it must be possible to see the line's current value and the full [audit] history of corrections that produced it.

Here are some **examples and expected numbers** to help you check your implementation:

Step	Event	Amount	Comment	Current value after this step
1	System calculates the line	—	—	\$1,000.00
2	Source data changes, system recalculates (no manual correction yet, so this is allowed)	—	—	\$1,050.00

3	Specialist adds a manual correction	−\$45.55	"Employee declined dental benefit; reversing deduction"	\$1,004.45
4	Source data changes again, system attempts to recalculate	—	(must be ignored — line already has a manual correction)	\$1,004.45
5	Specialist adds a second correction	+\$100.10	"Late correction: missed approved overtime bonus"	\$1,104.55
6	Specialist adds a third correction	−\$0.10	"Minor rounding adjustment"	\$1,104.45
7	Specialist adds a fourth correction	−\$0.20	"Second minor rounding adjustment"	\$1,104.25
8	Specialist adds a compensating correction, realizing step 7 was a mistake	+\$0.20	"Correcting mistake in adjustment #4"	\$1,104.45

Expected final audit history for this line:

Entry	Value
System value (frozen at step 3)	\$1,050.00
Adjustment 1	−\$45.55
Adjustment 2	+\$100.10
Adjustment 3	−\$0.10
Adjustment 4	−\$0.20
Adjustment 5	+\$0.20
Current (new) value	\$1,104.45

What we do expect to see:

- A solution written in easy to understand and modern PHP code
- Test coverage with PHPUnit
- A README file explaining how it works and any assumptions you've made
- Pushed to a public Github repo
- Using AI tools (ChatGPT, Claude, Copilot, etc.) while working on this task is welcome, and, moreover, highly encouraged

Email template

Hello {{CANDIDATE_FIRST_NAME}},

{{HR's greetings}}

We want you to know that we respect your skills and your time. This code test isn't crazy hard and it's not designed to see whether you're willing to sacrifice entire days of your life for a shot at the job.

It is designed to make sure that you can solve problems (even made-up ones!) and do so in a way that's easy for other developers to follow.

With that in mind, we recommend you spend 2-6 hours on it. This isn't a university assignment where you need to get 100%. This is a chance to show that your code isn't going to make other people on the team cry.

You're unlikely to be able to demonstrate everything in this list in time. Please consider it a jumping off point to show what you know:

- Modern PHP
- Good separation / encapsulation of concerns
- Small accurate interfaces / classes / commands / events / aggregate boundaries
- Dependency injection
- Source control, conventional / meaningful commits and essential comments

Our real codebase uses DDD + CQRS + Event Sourcing, but here you may use that approach or a different one (e.g. a simpler object-oriented design without ES). What matters is that the solution correctly and reliably implements the business rules above, and that you can justify your choice.

To avoid confusion, this is a data modeling exercise, not something that needs a UI or framework. Unit tests and something that you run from the command line are plenty and will give you more time to show us how you think about engineering rather than spending it on annoying wiring work!

If successful, you will progress to a technical interview where you can walk us through your thought process when completing the assignment.

Please check the following link for the assignment - {{LINK_TO_CODING_TEST}}

Good luck!